

MODULE 02 · DOMAIN 1 - AGENTIC ARCHITECTURE & ORCHESTRATION

Task Decomposition

Five patterns for splitting work up, and the one question that tells you which to use.

DOMAIN 1 of 5 EXAM WEIGHT 27% MODULE 2 of 20

Every decomposition pattern answers the same question: **who writes the list of subtasks, and when?**

The five patterns at a glance

PATTERN	WHO WRITES THE SUBTASK LIST	WHEN
Prompt chaining	You	Author time. Fixed order.
Routing	You, one list per category	Author time. Model picks the branch.
Parallelisation	You	Author time. All at once.
Orchestrator-workers	The model	Run time, from the input.
Evaluator-optimiser	You, but the count is open	Author time, repeated until good.

ON THE EXAM

The first three are workflows. The fourth is agentic. The fifth is a loop with a judge in it.

Prompt chaining

01

Step 1

Focused call. Output feeds the next.

02

Gate

Your code checks the output before continuing.

03

Step 2

A different, narrower job.

04

Step 3

Final output, assembled from the chain.

ON THE EXAM

Chaining trades latency for accuracy: more calls, each one easier. Use it when the steps are known and each is genuinely simpler alone.

The gate is the point

A programmatic check between steps, written by you, running every time.

- ◆ **It runs in your code, not the model's.** A schema check, a length check, a lookup, a regex.
- ◆ **It stops bad output propagating.** Step three never sees something step two got wrong.
- ◆ **It gives you provable coverage.** If the gate ran, the step ran. You can say so to an auditor.
- ◆ **An agent cannot offer this** without extra machinery, because the agent chooses whether a step happens at all.

ON THE EXAM

"We cannot tell whether a missing finding means clean or skipped" is a chaining answer. The gate is what makes it provable.

Routing

01

Classify

One cheap call decides which category the input is.

02

Dispatch

Your code sends it down the matching branch.

03

Handle

A specialised prompt or model, tuned for that category alone.

ON THE EXAM

Routing exists so that optimising one path cannot degrade another. That is the whole benefit.

Routing or one bigger prompt?

ROUTE

Distinct categories

- Categories are separable and stable
- Handling differs meaningfully per category
- Tuning one path must not affect others
- Cost or latency varies by category

ONE PROMPT

Blurred categories

- Inputs mix categories in one request
- Handling is broadly the same either way
- Misclassification would be expensive
- The classifier adds a hop for little gain

THE TIEBREAKER

Routing puts a classifier in front of everything. If it gets the category wrong, the right branch never runs.

Parallelisation

01

Split

You divide the work into independent pieces.

02

Run at once

All pieces execute concurrently.

03

Aggregate

Your code combines the results.

ON THE EXAM

The word that licenses parallelisation is independent. If piece two needs piece one's output, this is a chain wearing a disguise.

Two flavours of parallel

- ◆ **Sectioning.** Different pieces of the work, run side by side. Twelve precedents, twelve analyses.
- ◆ **Voting.** The *same* work, run several times, results compared or combined.
- ◆ **Voting buys recall.** Run a security scan three times with different framings and take the union.
- ◆ **Sectioning buys speed.** Voting buys confidence and costs more. They solve different problems.

ON THE EXAM

If the question is "this takes too long", you want sectioning. If it is "we keep missing things", you want voting.

How parallel delegation actually happens

- ◆ **The coordinator emits several Task calls in a single assistant response.** That is what makes them concurrent.
- ◆ **One Task call, wait, then another** is sequential, however it is described in the prompt.
- ◆ **Asking the model to "work in parallel"** changes nothing on its own. Concurrency is a property of the message, not the wording.
- ◆ **allowedTools must include Task** or the coordinator will reason about delegating and never do it.

ON THE EXAM

Sequential subagent calls that should be parallel is a common exam symptom. The fix is in the response shape.

Parallelism buys you **wall-clock time**. It does not buy you tokens, and it usually costs more of them.

Orchestrator-workers

01

Read the input

A central model examines the actual request.

02

Decide the subtasks

It works out what pieces this particular job needs.

03

Delegate

Workers run those pieces.

04

Synthesise

The orchestrator combines and judges the results.

ON THE EXAM

The subtask list is different for every input. That is the whole distinction from parallelisation.

Parallelisation or orchestrator-workers?

PARALLELISATION

Subtasks fixed in advance

- You wrote the split; it never varies
- Twelve precedents, twelve identical analyses
- Aggregation is code you control
- Fully predictable, easy to test

ORCHESTRATOR-WORKERS

Subtasks determined at run time

- The model decides what pieces are needed
- A simple query spawns one worker, a hard one six
- Synthesis is a judgement, not an append
- Flexible, and harder to reason about

THE TIEBREAKER

Could you write the subtask list before seeing the input? Yes means parallelisation. No means orchestrator-workers.

Evaluator-optimiser

01

Generate

One call produces a candidate.

02

Evaluate

A second call critiques it against stated criteria.

03

Revise

The critique feeds back into a new attempt.

04

Stop

When the evaluator passes it, or a cap is reached.

ON THE EXAM

This is the only pattern where the number of iterations is unknown at author time.
Always give it a cap.

When the critic earns its keep

- ◆ **You can state the criteria clearly.** If you cannot write down what good looks like, the evaluator has nothing to check.
- ◆ **Iteration measurably helps.** The second draft is genuinely better than the first, and you can show it.
- ◆ **Not when a deterministic check would do.** A schema validator or a test suite is cheaper and unarguable.
- ◆ **Not for the same model grading itself in the same context.** That agrees with itself far too often.

ON THE EXAM

Prefer the cheapest check that cannot be argued with. A model critic is the fallback when judgement is genuinely required.

When none of the five fit

The stem is telling you nobody can write the list.

- ◆ **"The engineer hasn't specified which modules to prioritise."** The subtask list depends on what exploration finds.
- ◆ **"Trace the error through routing, middleware, business logic and the database."** Where it lives decides where to look next.
- ◆ **"The codebase has eight hundred files and nobody here has read them."**
- ◆ **Then let the agent generate subtasks as it goes**, with tools, a turn limit, and something to verify against.

ON THE EXAM

Open-ended plus unfamiliar plus discovery-driven equals agentic. Do not force one of the five onto it.

The middle ground people forget

SHAPE	WHAT IT GIVES YOU
Fixed chain	Provable coverage. Blind to anything outside the step you defined.
Turn-limited agentic task	Freedom to follow a thread, with a hard ceiling on how far.
Unbounded agent	Maximum flexibility. No guarantee it terminates usefully.

ON THE EXAM

A per-file chain cannot see a cross-file bug. Giving the model tools and a turn cap can, without giving up control.

What decomposition costs you

- ◆ **Latency.** Every boundary is another round trip, unless the pieces run concurrently.
- ◆ **Tokens.** Each piece needs its own framing context, so the total grows.
- ◆ **Information at the seams.** Whatever step two does not pass on, step three cannot use.
- ◆ **Debugging.** Six components failing rarely is one system failing regularly.
- ◆ **Decompose because a step is genuinely easier alone**, not because the diagram looks more professional.

ON THE EXAM

Loss at the seams is the biggest one, and Domain 5 is largely about it.

Reading the stem: which pattern?

WHAT THE STEM SAYS	WHAT IT WANTS
"the same three steps every time"	Prompt chaining
"we can't tell if a step ran"	Prompt chaining, for the gate
"tuning one path degrades another"	Routing, or split prompts
"these tasks are independent"	Parallelisation, sectioning
"we keep missing genuine issues"	Parallelisation, voting
"query distribution is diverse and evolving"	Orchestrator-workers
"depends on what it discovers at each step"	Agentic, dynamic subtasks

ON THE EXAM

Read the stem for one of these before you read the options. Most questions plant exactly one.

Module 2 in five lines

- ◆ Ask who writes the subtask list and when. That single axis separates all five patterns.
- ◆ Chaining for fixed steps, and for the gate that proves a step ran.
- ◆ Independent means parallel. Sectioning for speed, voting for recall.
- ◆ Parallelisation splits a list you wrote. An orchestrator writes the list at run time.
- ◆ Parallelism buys wall-clock, never cost. Latency questions only.

ON THE EXAM

Seven worked scenarios next, and several of them offer three patterns at once.

A legal research system analyses court judgments. A coordinator delegates document analysis to a subagent.

For an ordinary case this takes about fifteen seconds. For a landmark judgment citing twelve precedents, the subagent works through them one after another and the whole analysis takes over three minutes. Users abandon the request.

The team wants the latency down, but the coordinator must still be able to monitor progress and debug individual precedent analyses when they go wrong.

What is the most effective change?

- | | |
|--|--|
| <p>A Have the single analysis subagent process all twelve precedents in one larger prompt</p> | <p>B Have the coordinator spawn parallel analysis subagents, each handling a subset, then aggregate</p> |
| <p>C Analyse the three most-cited precedents and note that the rest were omitted for time</p> | <p>D Move the analysis to the Message Batches API to reduce per-precedent cost</p> |

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 One larger prompt
Faster than twelve sequential calls, but you lose per-precedent visibility. The stem explicitly asked to keep it.
- B

 Parallel subagents, aggregate
Precedents are independent, so sectioning applies. Each stays a separate unit the coordinator can monitor.
- C

 Analyse only the top three
Fixes latency by not doing the work. A legal tool that silently skips precedents is worse than a slow one.
- D

 Batch API
Batch is for cost when nobody is waiting. Here a user is waiting, and batch is slower, not faster.

THE SIGNAL

Two constraints, and the second one does the work. "Must still monitor and debug individual analyses" is what eliminates A.

A production API returns intermittent five hundred errors. The cause could be in request routing, in middleware, in the business logic, or in the database layer, and nobody knows which.

An engineer asks an agent with codebase tools to find it. A colleague proposes hard-coding the investigation as four fixed steps: check routing, then middleware, then business logic, then the database.

What task decomposition approach is most effective here?

- ☐ A Have the agent generate investigation subtasks dynamically based on what it finds at each step
- ☐ B Run the four fixed layer checks as a prompt chain, in order
- ☐ C Run the four layer checks in parallel and aggregate the findings
- ☐ D Route the error to a specialised subagent based on the HTTP status code

◆ PAUSE AND DECIDE

What task decomposition approach is most effective here?

- A

Dynamic subtasks
Debugging is discovery-driven. A stack trace in middleware changes what you look at next, and no author can predict that.
- B

Fixed four-step chain
Checks four layers in isolation and cannot follow a thread between them. It also cannot stop early when it finds the cause.
- C

Four checks in parallel
Faster and equally blind. Parallel makes a decomposition quicker, not smarter.
- D

Route by status code
A five hundred says something failed, not where. There is nothing here to classify on.

THE SIGNAL

Compare with module 1, scenario 2. That PR review was three fixed independent checks, so chaining won. Here the next step depends on the last, so it loses.

A research coordinator delegates to a web search subagent and a document analysis subagent. Neither needs the other's output; they read different sources and their findings are merged afterwards.

Traces show the coordinator emits a Task call for web search, waits for the result, then emits a Task call for document analysis. Total time is the sum of both. The system prompt already instructs the coordinator to run independent research in parallel.

How should the system be changed to run these concurrently?

- ☐ A Strengthen the system prompt instruction, adding that parallel execution is required, not optional
- ☐ B Structure the coordinator to emit both Task calls in a single response message
- ☐ C Merge the two subagents into one that searches and analyses in a single pass
- ☐ D Raise max_turns so the coordinator has room to issue both calls

◆ PAUSE AND DECIDE

How should the system be changed to run these concurrently?

A Strengthen the instruction

The instruction is already there and is being followed loosely. Concurrency is a property of the message shape, not of emphasis.

C Merge into one subagent

Removes the parallelism problem by removing the parallelism. It also destroys the context isolation the split was for.

B Both Task calls in one response

Two tool calls in a single assistant response execute together. That is the mechanism.

D Raise max_turns

It issued both calls. Turns were never the constraint.

THE SIGNAL

"The system prompt already instructs it" is the planted detail. When a prompt instruction is already present and not working, stop reaching for a firmer prompt.

A research assistant serves a mixed workload. Some queries are one-line factual lookups. Others need web search, document analysis and synthesis across a dozen sources. New kinds of query keep appearing as users find new uses for it.

Today every query runs the full four-subagent pipeline. Simple lookups take forty seconds and cost as much as complex ones.

What is the most effective way to handle the varying complexity?

- ☐ A Add a length threshold: queries under fifteen words skip straight to a single call
- ☐ B Build two fixed pipelines, simple and complex, and route by keyword match
- ☐ C Have the coordinator assess each query and decide which subagents to invoke for it
- ☐ D Run all four subagents in parallel so even complex queries return quickly

◆ PAUSE AND DECIDE

What is the most effective way to handle the varying complexity?

- A

 Length threshold
Length does not track complexity. "Compare all EU AI Act drafts" is six words and needs the full pipeline.
- B

 Two pipelines, keyword routing
Genuinely good when categories are stable. The stem says new query types keep appearing, so the keyword list is obsolete on arrival.
- C

 Coordinator decides per query
Diverse and evolving input is the orchestrator case. The subtask list is produced from the query itself.
- D

 All four in parallel
Cuts latency and not cost, and still runs three subagents a lookup never needed.

THE SIGNAL

"Diverse and evolving" is orchestrator language. Anything you hard-code today is wrong by next month, which is what rules out routing.

An automated review runs on each changed file in a pull request: one call per file, with the file contents and the team's standards.

It performs well on single-file issues. But an evaluation finds that cross-file bugs — a caller not updated after a signature change, a contract broken between two modules — account for thirty-five per cent of production incidents from reviewed PRs, and the review catches almost none of them.

What is the most effective change to the review design?

- | | |
|---|---|
| <p>☐ A Include every changed file's contents in each per-file review call</p> | <p>☐ B Redesign it as a turn-limited agentic task where the model can read files and search the codebase</p> |
| <p>☐ C Add a system prompt instruction to consider cross-file impacts when reviewing each file</p> | <p>☐ D Add a second per-file pass focused specifically on cross-file concerns</p> |

◆ PAUSE AND DECIDE

What is the most effective change to the review design?

- A

 All files in every call
Helps a little, then hits the context window on a large PR. It also misses callers that were not changed.
- B

 Turn-limited agentic task
The model can follow a signature to its callers, including files the PR never touched. Bounded by turns, not by structure.
- C

 Prompt it to consider cross-file impact
The call physically cannot see the other file. No instruction fixes an information problem.
- D

 Second per-file pass
Twice the cost, same blindness. The unit of work is still one file.

THE SIGNAL

The decomposition is structurally blind, not badly prompted. When the unit of work cannot contain the bug, change the unit of work.

A research coordinator has just completed a full pipeline and holds all the findings in its context. It returns a report.

The user then asks three short follow-ups: "summarise the second section in two sentences", "which source said that", and "reword the conclusion for a non-technical reader".

Each follow-up currently spawns a fresh synthesis subagent, which re-reads the findings before answering. Users complain the follow-ups are slower than the original report was.

What is the most effective way to improve follow-up response time?

- | | |
|---|---|
| <p>A Have the coordinator answer straightforward follow-ups directly from its existing context</p> | <p>B Cache the synthesis subagent's output so repeat requests are served from cache</p> |
| <p>C Spawn the synthesis subagents for all three follow-ups in parallel</p> | <p>D Pass a condensed summary rather than the full findings when spawning the subagent</p> |

◆ PAUSE AND DECIDE

What is the most effective way to improve follow-up response time?

- A

 Coordinator answers directly
It already holds the findings. Spawning a subagent to re-read what you are holding is pure overhead.
- B

 Cache subagent output
Each follow-up is a different question. There is nothing repeated to hit in the cache.
- C

 Parallel subagents
Three unnecessary subagents concurrently is still three unnecessary subagents. Cheaper in wall-clock, not in principle.
- D

 Condensed summary
Makes the wrong operation cheaper, and loses the detail "which source said that" needs.

THE SIGNAL

Not every question about a multi-agent system has a multi-agent answer. Reserve subagents for work needing isolation or fresh capacity.

A security scan reviews merged code for injection flaws, unsafe deserialisation and secret leakage. A single well-tuned pass finds about seventy per cent of the issues a human auditor later confirms.

The team's priority is recall: a missed vulnerability is expensive, and they have spare reviewer capacity to triage false positives. Latency is not a concern — the scan runs nightly.

What change most improves recall?

- ☐ A Run the scan several times with different framings and take the union of findings
- ☐ B Split the scan into three sequential steps, one per vulnerability class
- ☐ C Add an evaluator pass that reviews the findings and removes low-confidence ones
- ☐ D Route each file to a specialised scanner based on its language

◆ PAUSE AND DECIDE

What change most improves recall?

- A

 Several runs, take the union
Voting. Different framings surface different issues, and the union is strictly higher recall.
- B

 Three sequential steps
Sectioning by class. Useful, but each pass still misses what one pass misses within its class.
- C

 Evaluator removes low-confidence findings
Improves precision by discarding findings. It moves recall in the wrong direction.
- D

 Route by language
Specialisation helps, but nothing suggests language is where the misses come from.

THE SIGNAL

Three enabling details: recall matters, spare triage capacity, latency irrelevant.
Together they license the expensive, noisy option.